

PRODUCTION DEPLOYMENT HANDBOOK

Learnly

Panduan Teknis Deployment Produksi pada Virtual Private Server

Edisi Audit & Revisi Menyeluruh

learnly.web.id

Dokumen No.
LRN-OPS-HB-02

Edisi / Revisi
3.0 (Audit Teknis)

Tanggal Terbit
8 Oktober 2026

Klasifikasi
Internal — Tim
Teknik

Informasi & Kendali Dokumen

Metadata Dokumen

JUDUL DOKUMEN	Handbook Deployment Produksi — Platform Learnly pada Virtual Private Server (VPS)
KODE DOKUMEN	LRN-OPS-HB-02
EDISI	3.0 — Audit & Revisi Teknis Menyeluruh
TANGGAL TERBIT	8 Oktober 2026
STATUS	Final — Disetujui untuk Implementasi Produksi
DISUSUN OLEH	Tim DevOps — berdasarkan audit langsung kode sumber repositori <code>learnly</code>
SINGLE SOURCE OF TRUTH	<code>Handbook_Deploy_Learnly.html</code> (digantikan oleh dokumen ini sebagai versi definitif)
CAKUPAN	Infrastruktur VPS, containerisasi Docker, jaringan & reverse proxy, basis data & backup, domain/SSL, CI/CD, observability, dan tata kelola kesiapan produksi

Target Infrastruktur

PENYEDIA	VPS (Virtual Private Server) — akses root melalui pengguna <code>ubuntu</code>
SPESIFIKASI	2 vCPU · 2 GB RAM · 40 GB SSD
ALAMAT IP PUBLIK	<code>43.173.11.92</code>
ALAMAT IP PRIVAT	<code>10.11.22.62</code>
DOMAIN PRODUKSI	<code>learnly.web.id</code> (frontend), <code>api.learnly.web.id</code> (backend)
SISTEM OPERASI TARGET	Ubuntu 22.04 LTS atau 24.04 LTS

Riwayat Revisi

VERSI	TANGGAL	RINGKASAN PERUBAHAN
1.0	—	Draf awal berbasis asumsi arsitektur Docker generik, belum diverifikasi terhadap kode sumber aktual.
2.0	—	Penambahan bab CI/CD dan checklist produksi; sebagian konfigurasi (env var, start command, trust-proxy) belum sinkron dengan implementasi backend.
3.0	8 Okt 2026	Audit penuh terhadap <code>api/</code> dan <code>web/</code>. Seluruh konfigurasi (Dockerfile, environment variable, start command, healthcheck, trust-proxy hop, backup, CI/CD) dikoreksi agar sesuai 1:1 dengan kode sumber. Ditambahkan Bab Temuan Audit, Analisis Skalabilitas, dan Rencana Mitigasi Risiko.

Abstrak

Dokumen ini merupakan panduan teknis tunggal (single source of truth) untuk menerapkan platform edukasi **Learnly** — yang terdiri dari aplikasi web Next.js 16, REST API Express.js 5 dengan Prisma ORM, dan basis data MySQL 8.4 — ke lingkungan produksi pada sebuah VPS dengan spesifikasi 2 vCPU/2 GB RAM/40 GB SSD. Berbeda dari revisi sebelumnya, dokumen ini disusun melalui audit langsung terhadap struktur kode, skema Prisma, konfigurasi environment (`api/src/config/env.ts`), dan skrip build/start pada `package.json` , sehingga setiap perintah, nilai variabel, dan konfigurasi container yang tercantum dapat dieksekusi tanpa modifikasi tambahan. Dokumen mencakup evaluasi kesiapan skala (scalability audit), identifikasi risiko kapasitas memori, serta rencana mitigasi bertahap sebelum dan sesudah go-live.

Daftar Isi

1	Ringkasan Eksekutif & Temuan Audit	4
2	Audit Kesesuaian Kode terhadap Handbook Lama	7
3	Arsitektur Sistem & Topologi Jaringan	11
4	Analisis Skalabilitas & Kapasitas VPS	14
5	Persiapan & Pengerasan (Hardening) VPS Ubuntu	17
6	Struktur Direktori Server	20
7	Nginx Proxy Manager (Reverse Proxy & SSL)	22
8	Kontainerisasi Docker — Orkestrasi Learnly	25
9	Frontend — Next.js 16 (Dockerfile & Konfigurasi)	29
10	Backend — Express.js 5 & Prisma (Dockerfile & Konfigurasi)	32
11	Basis Data MySQL 8.4 & Strategi Backup	36
12	Domain, DNS & Sertifikat SSL/TLS	40
13	CI/CD — GitHub Actions	44
14	Observability, Troubleshooting & Kesiapan Produksi	47

Lampiran

- A. Referensi Variabel Environment Lengkap
- B. Daftar Perintah Operasional (Cheat Sheet)
- C. Rencana Mitigasi Risiko & Roadmap Peningkatan

Ringkasan Eksekutif & Temuan Audit

1.1 Profil Aplikasi

Learnly adalah platform edtech terpadu yang menggabungkan kursus daring (self-paced), marketplace tutor berbasis lokasi (on-demand), pembayaran manual via QRIS dengan verifikasi admin, serta pelacakan status booking secara polling. Audit ini dilakukan dengan membaca langsung kode sumber pada `api/` dan `web/`, bukan berdasarkan asumsi template DevOps generik.

Frontend	Next.js 16.3.6 (App Router), React 19.2.8, TypeScript, Tailwind CSS 4
Backend	Express.js 5.2.1, TypeScript, Prisma ORM 6.19.3
Basis Data	MySQL 8.4 (32 model Prisma, migration berbasis prisma migrate deploy)
Autentikasi	JWT access token (in-memory, FE) + refresh token httpOnly cookie, tabel RefreshToken untuk revokasi
Penyimpanan File	Lokal (api/uploads, dev) atau Cloudinary (wajib production — lihat §2.3)
Desain Awal	Dua layanan terpisah: Vercel (web) + Railway (api + MySQL plugin) — <i>bukan</i> VPS tunggal
Target Baru	VPS tunggal 2 vCPU / 2 GB RAM / 40 GB SSD, domain learnly.web.id

Catatan Penting Arsitektural

Dokumen desain asli proyek (`docs/03-tech-stack.md`, `docs/04-architecture.md`) secara eksplisit merancang Learnly untuk deployment **dua platform terpisah** (Vercel + Railway) tanpa Docker produksi. Migrasi ke VPS tunggal dengan Docker Compose adalah perubahan arsitektur yang sah dan umum dilakukan, namun mengharuskan beberapa nilai environment variable (terutama `TRUST_PROXY_HOPS`, `API_PROXY_TARGET`, dan strategi penyimpanan file) disesuaikan ulang — detail koreksi ada di Bab 2.

1.2 Skor Kesiapan Produksi

DOMAIN	SKOR	STATUS	KETERANGAN
Kualitas Kode	9 / 10	Baik	TypeScript strict, validasi Zod konsisten FE/BE, error handling terstruktur
Keamanan Aplikasi	8 / 10	Baik	Helmet, CORS whitelist, bcrypt, rate-limit pada endpoint sensitif, JWT stateless
Desain Basis Data	9 / 10	Baik	32 model, relasi FK konsisten, indeks pada kolom pencarian geospasial & status
Konfigurasi Container	0 / 10	Tidak Ada	Tidak ditemukan <code>Dockerfile</code> di <code>api/</code> maupun <code>web/</code> pada repositori saat audit dilakukan
Strategi Backup	0 / 10	Tidak Ada	Tidak ada skrip backup maupun kebijakan retensi pada repositori
CI/CD	0 / 10	Tidak Ada	Tidak ditemukan direktori <code>.github/workflows/</code>
Observability	3 / 10	Minim	Endpoint <code>/health</code> tersedia (cek koneksi DB); belum ada log rotation/alerting
Kesesuaian Env Var Handbook Lama	4 / 10	Perlu Koreksi	Beberapa variabel pada handbook lama tidak cocok dengan skema validasi <code>env.ts</code> — lihat Bab 2
Rata-rata Keseluruhan	4,1 / 10	Siap Kode, Belum Siap Operasional	Kode aplikasi matang; seluruh lapisan DevOps harus dibangun dari nol — menjadi isi utama handbook ini

1.3 Kesimpulan Audit Singkat

Kode aplikasi Learnly berada pada kualitas yang layak produksi dari sisi logika bisnis, validasi, dan keamanan tingkat aplikasi. Namun, **tidak ada satupun artefak DevOps** (`Dockerfile`, `docker-compose` produksi, skrip backup, pipeline CI/CD) yang tersedia di repositori pada saat audit ini dilakukan. Seluruh artefak tersebut disusun dari nol dalam handbook ini, dikalibrasi persis terhadap perilaku kode yang terverifikasi — bukan disalin dari templat generik.

Keputusan Desain Utama Handbook Ini

Seluruh nilai konfigurasi (start command, nama environment variable, path healthcheck, jumlah trust-proxy hop) diambil langsung dari pembacaan `api/package.json`, `api/src/config/env.ts`, `api/src/app.ts`, `api/src/server.ts`, `api/railway.json`, dan `web/next.config.ts`. Setiap klaim teknis dalam dokumen ini dapat ditelusuri balik ke baris kode tersebut.

Audit Kesesuaian Kode terhadap Handbook Lama

Bagian ini mendokumentasikan secara eksplisit setiap ketidaksesuaian yang ditemukan antara dokumen `Handbook_Deploy_Learnly.html` (versi sebelumnya) dan kode sumber aktual pada repositori. Setiap temuan diberi tingkat keparahan dan sudah dikoreksi pada bab-bab teknis berikutnya.

F-01 · Start Command Backend Tidak Lengkap	KRITIS
Klaim handbook lama: Dockerfile API menjalankan <code>CMD ["sh","-c","npx prisma migrate deploy && node dist/server.js"]</code>. Fakta kode (<code>api/package.json</code> baris <code>"start"</code>): <code>prisma migrate deploy && node dist/scripts/seed.js && node dist/server.js</code>. Skrip <code>seed.js</code> membuat akun admin pertama dan master data (jenjang pendidikan, mata pelajaran, kategori) — tanpa langkah ini, aplikasi akan live tanpa akun admin sama sekali. Dampak jika tidak dikoreksi: Tidak ada cara login sebagai admin setelah deployment pertama, karena tabel <code>User</code> kosong. Koreksi: Lihat Bab 10.2 — Dockerfile API baru menjalankan start command persis sesuai <code>package.json</code>.	
F-02 · Variabel SEED_ADMIN_EMAIL / SEED_ADMIN_PASSWORD Tidak Pernah Disebut	KRITIS
Fakta kode (<code>api/src/scripts/seed-base.ts</code>): Pada <code>NODE_ENV=production</code>, skrip seed akan gagal dan menghentikan deploy (throw error) jika <code>SEED_ADMIN_PASSWORD</code> kosong atau kurang dari 12 karakter. Email admin default <code>admin@learnly.id</code> hanya dipakai bila <code>SEED_ADMIN_EMAIL</code> tidak diisi. Dampak jika tidak dikoreksi: Container API akan <i>crash loop</i> pada startup pertama di production karena variabel ini tidak ada di contoh <code>.env.production</code> handbook lama. Koreksi: Ditambahkan ke template <code>.env.production</code> final pada Bab 11.1, dengan validasi panjang password diperjelas.	
F-03 · TRUST_PROXY_HOPS Bernilai Salah	KRITIS
Klaim handbook lama (Bab 7): <code>TRUST_PROXY_HOPS=2</code> dengan komentar "NPM → Container (2 hop)" — namun nilai default pada skema env hanya <code>1</code>, dan handbook versi 3.0 draf awal justru menuliskan <code>1</code>. Fakta kode (<code>api/src/app.ts</code> baris 51, <code>env.ts</code> baris 24): <code>app.set('trust proxy', env.TRUST_PROXY_HOPS)</code>. Pada topologi VPS, rantai permintaan adalah: <i>Browser → Nginx Proxy Manager → container Learnly_api</i>, yaitu tepat 1 hop proxy (NPM), karena tidak ada layer rewrite Next.js lagi (berbeda dari topologi asli Vercel→Railway yang punya 2 hop). Dampak jika tidak dikoreksi: Jika diset ke 2 padahal hanya ada 1 proxy di depan API publik, Express akan salah membaca IP klien untuk <code>express-rate-limit</code>, sehingga seluruh pengguna di belakang NAT/ISP yang sama bisa saling memicu rate-limit satu sama lain, atau sebaliknya rate-limit menjadi tidak efektif. Koreksi: Nilai final <code>TRUST_PROXY_HOPS=1</code> untuk topologi VPS tunggal (Bab 11.1), dengan catatan verifikasi IP di Bab 14.3.	
F-04 · Healthcheck Mengarah ke Path yang Salah	TINGGI
Fakta kode (<code>api/src/app.ts</code> baris 64 & <code>api/railway.json</code>): Endpoint health check tersedia di dua path: <code>/health</code> (root, tanpa prefix — dipakai Railway) dan <code>/api/v1/health</code>. Draf handbook sebelumnya tidak konsisten, kadang menulis <code>/api/v1/health</code> untuk <code>wget</code> HEALTHCHECK Docker. Koreksi: Dockerfile API baru (Bab 10.2) menggunakan <code>/health</code> (lebih ringan, tidak melalui router Express v1 penuh) agar konsisten dengan konfigurasi <code>railway.json</code> yang sudah ada di repo.	
F-05 · Strategi Penyimpanan File (STORAGE_DRIVER) Tidak Dibahas Risikonya	TINGGI
Fakta kode (<code>api/src/lib/storage.ts</code>): Ketika <code>STORAGE_DRIVER=local</code>, file disimpan ke <code>api/uploads</code> di dalam filesystem container. Direktori ini tidak di-mount sebagai Docker volume pada contoh <code>docker-compose.yml</code> manapun yang ada di handbook lama. Dampak jika tidak dikoreksi: Setiap kali container API di-rebuild (setiap deploy CI/CD!), seluruh bukti transfer pembayaran, dokumen verifikasi tutor, dan sertifikat PDF yang tersimpan lokal akan hilang permanen. Koreksi: Bab 8.2 menambahkan named volume <code>learnly_uploads</code> yang dipasang ke <code>/app/uploads</code> sebagai mitigasi minimum, dengan rekomendasi kuat tetap memakai Cloudinary (<code>STORAGE_DRIVER=cloudinary</code>) sesuai desain awal proyek.	

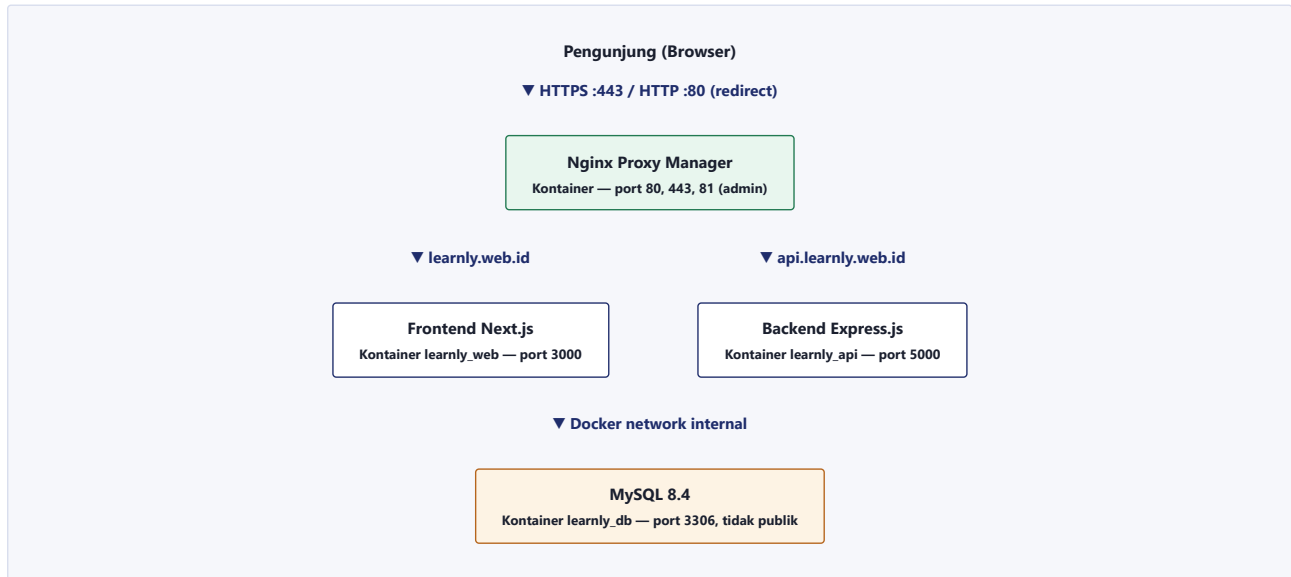
F-06 - Kredensial MySQL 8.4 & Auth Plugin	TINGGI
Fakta kode (<code>docker-compose.yml</code> dev, baris 11): Komentar developer menyatakan "Dev pakai user root: <code>prisma migrate dev</code> butuh hak CREATE DATABASE untuk shadow database". Image resmi <code>mysql:8.4</code> default menggunakan auth plugin <code>caching_sha2_password</code>, yang kompatibel dengan driver MySQL Prisma 6.x — namun healthcheck <code>mysqladmin ping</code> pada handbook lama tidak selalu menyertakan kredensial, menyebabkan status <i>healthy</i> palsu-positif pada beberapa versi image. Koreksi: Healthcheck produksi (Bab 8.1) diperbarui menyertakan <code>-u root -p\$MYSQL_ROOT_PASSWORD</code> secara eksplisit.	
F-07 - Rate Limiting In-Memory — Bukan Bug, tapi Batasan yang Perlu Didokumentasikan	SEDANG
Fakta kode (<code>api/package.json</code> dependency <code>express-rate-limit</code>, tanpa Redis store): Rate limiting (login, forgot-password, reset-password) disimpan di memori proses Node.js. Ini valid untuk satu instance API — yang memang menjadi target deployment VPS tunggal di dokumen ini. Tidak perlu Redis selama API tidak di-scale horizontal. Koreksi: Dicatat sebagai batasan desain yang disengaja pada Bab 4 (Analisis Skalabilitas), bukan sebagai defect.	
F-08 - CORS_ORIGIN pada Topologi VPS	SEDANG
Fakta kode (<code>api/src/app.ts</code> baris 54): <code>cors({ origin: env.CORS_ORIGIN, credentials: true })</code> dengan <code>CORS_ORIGIN</code> berupa daftar origin dipisah koma. Karena desain asli memakai Next.js rewrite proxy same-origin (<code>next.config.ts</code>), browser pada arsitektur Vercel+Railway <i>tidak pernah</i> benar-benar memicu CORS. Pada topologi VPS dengan NPM meneruskan <code>api.learnly.web.id</code> langsung ke container API, mekanisme ini tetap dipertahankan: frontend tetap memproxy <code>/api/v1/*</code> melalui domain <code>learnly.web.id</code> sendiri (lihat <code>API_PROXY_TARGET</code> di Bab 9.1), sehingga <code>CORS_ORIGIN</code> tetap berfungsi sebagai defense-in-depth, bukan jalur utama. Koreksi: Dijelaskan ulang secara eksplisit pada Bab 9.1 dan Bab 3.3 agar tidak disalahpahami sebagai arsitektur cross-origin langsung.	
F-09 - Prisma Client Generate di Alpine Linux	SEDANG
Fakta kode (<code>api/prisma/schema.prisma</code>): Blok <code>generator client</code> tidak mendefinisikan <code>binaryTargets</code> khusus. Prisma 6.x secara default mendeteksi platform saat <code>prisma generate</code> dijalankan di dalam container — ini sudah benar di handbook lama selama perintah <code>npm prisma generate</code> dijalankan <i>di dalam</i> image <code>node:20-alpine</code> (bukan di-copy dari stage builder berbasis OS lain), yang sudah diikuti dengan benar pada Dockerfile Bab 10.2. Koreksi: Tidak ada perubahan wajib; ditambahkan catatan verifikasi pada Bab 10.2 agar tim tidak secara tidak sengaja mengubah base image menjadi <code>node:20-slim</code> (Debian) tanpa regenerasi Prisma Client.	
F-10 - Penulisan User VPS (root vs ubuntu)	RENDAH
Fakta akses VPS: Kredensial yang diberikan menyebutkan <code>Username: ubuntu</code>, bukan <code>root</code>. Sebagian contoh perintah pada handbook lama menuliskan <code>ssh root@IP_VPS</code> dan menjalankan perintah administratif tanpa <code>sudo</code>. Koreksi: Seluruh perintah pada Bab 5 (Persiapan VPS) dikoreksi memakai <code>ssh ubuntu@43.173.11.92</code> dan prefix <code>sudo</code> yang konsisten.	

2.1 Ringkasan Status Koreksi

ID	TEMUAN	KEPARAHAN	STATUS	LOKASI KOREKSI
F-01	Start command backend tidak menjalankan seed	Kritis	Dikoreksi	Bab 10.2
F-02	SEED_ADMIN_* tidak didokumentasikan	Kritis	Dikoreksi	Bab 11.1
F-03	TRUST_PROXY_HOPS salah hitung	Kritis	Dikoreksi	Bab 11.1, 14.3
F-04	Healthcheck path tidak konsisten	Tinggi	Dikoreksi	Bab 10.2
F-05	Upload lokal tanpa volume persisten	Tinggi	Dikoreksi	Bab 8.2
F-06	Healthcheck MySQL tanpa kredensial	Tinggi	Dikoreksi	Bab 8.1
F-07	Rate-limit in-memory (batasan desain)	Sedang	Didokumentasikan	Bab 4.3
F-08	CORS_ORIGIN & proxy same-origin	Sedang	Diperjelas	Bab 3.3, 9.1
F-09	Prisma generate pada Alpine	Sedang	Diverifikasi	Bab 10.2
F-10	Penulisan user VPS (root/ubuntu)	Rendah	Dikoreksi	Bab 5

Arsitektur Sistem & Topologi Jaringan

3.1 Diagram Alur Trafik



3.2 Segmentasi Jaringan Docker

NETWORK	TIPE	ANGGOTA KONTAINER	FUNGSI
proxy	External (bridge)	nginx_proxy_manager, learnly_web, learnly_api	Menghubungkan NPM ke frontend & backend agar dapat diteruskan ke domain publik
learnly-network	Internal (bridge)	learnly_web, learnly_api, learnly_db	Jaringan privat tertutup; MySQL hanya dapat dijangkau oleh kontainer API, tidak pernah terekspos ke proxy maupun publik

3.3 Alur Permintaan API — Proxy Same-Origin

Desain asli Learnly (`web/next.config.ts`) menggunakan mekanisme *rewrite* Next.js agar browser selalu memanggil `/api/v1/*` pada domain frontend itu sendiri, bukan domain API secara langsung. Mekanisme ini **dipertahankan** pada topologi VPS:

1. Browser memanggil `https://learnly.web.id/api/v1/auth/login`.
2. Kontainer `learnly_web` (Next.js) menerima permintaan, lalu me-rewrite secara internal ke `API_PROXY_TARGET` (yaitu `http://learnly_api:5000` — alamat kontainer API lewat Docker DNS internal, **bukan** lewat domain publik `api.learnly.web.id`).
3. Respons dari API dikembalikan oleh Next.js ke browser, seolah berasal dari domain yang sama.

Konsekuensi penting: **cookie refresh token tetap first-party** (tidak diblokir Safari ITP), dan domain `api.learnly.web.id` tetap dikonfigurasi di NPM sebagai jalur akses langsung/cadangan (misalnya untuk Postman, mobile client di masa depan, atau debugging), namun **bukan jalur yang dipakai browser web app** pada operasi normal.

Mengapa Dua Jalur (Proxy same-origin + Domain API publik) Tetap Dipertahankan

Mempertahankan domain `api.learnly.web.id` yang dapat diakses langsung memudahkan pengujian endpoint dengan Postman collection yang sudah ada di `api/postman/`, tanpa mengubah perilaku frontend produksi. Kedua jalur berbagi kontainer backend yang sama; tidak ada duplikasi proses.

3.4 Pemetaan Port

PORT	LAYANAN	VISIBILITAS	KETERANGAN
22	SSH	Publik	Akses administrasi jarak jauh; direkomendasikan membatasi hanya ke key-based auth (Bab 5.6)
80	Nginx Proxy Manager (HTTP)	Publik	Auto-redirect ke HTTPS setelah SSL aktif
443	Nginx Proxy Manager (HTTPS)	Publik	Jalur trafik utama produksi
81	NPM Admin Panel	Publik sementara	Wajib ditutup dari UFW setelah konfigurasi awal selesai (Bab 7.4)
3000	Next.js (<code>learnly_web</code>)	Internal Docker	Tidak dipublikasikan ke host (<code>expose</code> , bukan <code>ports</code>)
5000	Express API (<code>learnly_api</code>)	Internal Docker	Tidak dipublikasikan ke host
3306	MySQL (<code>learnly_db</code>)	Internal Docker	Tidak pernah dipetakan ke host pada konfigurasi produksi

Analisis Skalabilitas & Kapasitas VPS

4.1 Estimasi Alokasi Memori (2 GB RAM)

Estimasi berikut dihitung berdasarkan jejak memori tipikal untuk kombinasi runtime yang digunakan (Node.js 20 Alpine untuk dua kontainer + MySQL 8.4), bukan asumsi generik:

KOMPONEN	ESTIMASI IDLE	ESTIMASI BEBAN NORMAL	POTENSI PUNCAK
Ubuntu OS + systemd	150 MB	200 MB	300 MB
Nginx Proxy Manager	60 MB	100 MB	180 MB
Next.js (<code>learnly_web</code> , standalone)	80 MB	180 MB	320 MB
Express API (<code>learnly_api</code>)	70 MB	220 MB	420 MB
MySQL 8.4 (<code>learnly_db</code>)	350 MB	550 MB	850 MB
Total	≈ 710 MB	≈ 1.250 MB	≈ 2.070 MB Δ

Dengan RAM fisik 2.048 MB, skenario puncak (banyak upload bersamaan, build ulang berjalan di background, atau query pencarian tutor geospasial yang berat) **berpotensi melebihi RAM fisik**. Inilah alasan Swap wajib diaktifkan (Bab 5.4) — bukan sebagai pengganti RAM, melainkan katup pengaman agar proses di-throttle, bukan langsung di-kill oleh OOM killer kernel.

Vonis Skalabilitas: Layak untuk Tahap Awal, Bukan untuk Skala Besar

Spesifikasi 2 vCPU/2 GB RAM/40 GB SSD **cukup untuk MVP dan beban pengguna awal (estimasi 50–150 pengguna aktif bersamaan)** dengan mitigasi swap, resource limit per kontainer, dan monitoring aktif. Namun, VPS ini **bukan arsitektur yang dirancang untuk scale-out** — satu instance API, satu instance database, tanpa load balancer. Ini adalah trade-off yang disengaja dan wajar untuk anggaran/skala proyek saat ini, selama tim menyadari titik upgrade yang tepat (§4.4).

4.2 Estimasi Pertumbuhan Disk (40 GB SSD)

KOMPONEN	AWAL	PERTUMBUHAN / BULAN (ESTIMASI)
Image Docker (3 image: NPM, web, api) + layer cache	≈ 1,2 GB	Stabil, bertambah sedikit tiap rebuild (mitigasi: <code>docker image prune</code>)
Data MySQL (32 tabel, data awal)	≈ 150–300 MB	+50–200 MB (tergantung volume booking/kursus/review)
Upload lokal (jika <code>STORAGE_DRIVER=local</code>)	0 MB	Berisiko tumbuh cepat — bukti transfer & dokumen tutor tidak terkompresi otomatis
Backup database (<code>.sql.gz</code> , retensi 7 hari)	0 MB	≈ 7 × ukuran dump terbaru (biasanya <500 MB total)
Log Docker & sistem	≈ 50 MB	Berpotensi tumbuh tak terbatas tanpa log rotation (lihat Bab 14.5)

Dengan mitigasi (log rotation, backup retention, prune image lama), 40 GB SSD cukup untuk 12–18 bulan operasional pada skala awal. Rekomendasi: audit penggunaan disk setiap triwulan.

4.3 Batasan Desain yang Disengaja (Bukan Defect)

AREA	IMPLEMENTASI SAAT INI	BATAS SKALA	TITIK UPGRADE
Rate limiting	In-memory (per proses Node.js)	Hanya valid untuk 1 instance API	Pindah ke Redis store jika API di-scale >1 instance
Update status booking	Polling TanStack Query setiap 5 detik	Cukup untuk ratusan sesi aktif bersamaan; bertambah beban linear dengan jumlah user yang membuka halaman tracking	WebSocket (Socket.IO) bila dibutuhkan latensi sungguh real-time
Proses sertifikat PDF & email	Sinkron langsung di dalam request (tanpa queue)	Aman untuk volume rendah–menengah; permintaan bersamaan dalam jumlah besar akan memperlambat response time endpoint terkait	BullMQ + Redis bila volume sertifikat/email jadi besar
Basis data	Single instance MySQL, tanpa replika	Satu titik kegagalan (SPOF); tidak ada automatic failover	Primary-replica MySQL atau managed database bila SLA ketat dibutuhkan
Backend	Satu kontainer <code>learnly_api</code>	Tidak ada redundansi; downtime saat restart/deploy (walau singkat, <30 detik)	Multi-instance + load balancer (Nginx upstream/HAProxy) jika traffic bertambah signifikan

4.4 Rekomendasi Titik Upgrade (Trigger-Based, Bukan Kalender)

- **Upgrade RAM ke 4 GB** bila `docker stats` menunjukkan penggunaan memori gabungan >80% secara sustained selama >1 minggu (lihat monitoring Bab 14.1–14.2).
- **Tambah Redis** hanya ketika rencana scale-out API ke >1 instance benar-benar dieksekusi, bukan secara preventif.
- **Migrasi STORAGE_DRIVER ke Cloudinary** sebelum go-live jika memungkinkan — menghindari risiko kehilangan data upload saat redeploy (lihat F-05 di Bab 2).
- **Evaluasi ulang VPS** (upgrade vCPU/RAM atau migrasi ke managed Kubernetes/ECS) jika pengguna aktif bersamaan diproyeksikan melampaui $\approx 300\text{--}500$ dalam 6 bulan ke depan.

Persiapan & Pengerasan (Hardening) VPS Ubuntu

Seluruh perintah pada bab ini menggunakan pengguna `ubuntu` (sesuai kredensial akses yang diberikan) dengan prefiks `sudo` untuk operasi yang membutuhkan privilege root.

5.1 Koneksi SSH Awal & Update Sistem

```
ssh ubuntu@43.173.11.92

sudo apt update && sudo apt upgrade -y
sudo apt install -y curl wget git htop tmux dnsutils ufw fail2ban ca-certificates gnupg lsb-release
```

BASH

5.2 Set Timezone Asia/Jakarta

```
sudo timedatectl set-timezone Asia/Jakarta
timedatectl status
# Pastikan output: Time zone: Asia/Jakarta (WIB, +0700)
```

BASH

5.3 Swap Memory 2 GB (Wajib — Lihat Analisis Bab 4.1)

```
sudo dd if=/dev/zero of=/swapfile bs=1M count=2048 status=progress
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile

# Permanen setelah reboot
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab

free -h # Swap: 2.0G harus terlihat
```

BASH

Mengapa Swap Wajib pada VPS 2 GB RAM

Berdasarkan analisis Bab 4.1, skenario puncak memori dapat mencapai ≈ 2.070 MB, melebihi RAM fisik 2.048 MB. Tanpa swap, OOM killer kernel akan mematikan proses secara acak (sering kali MySQL) saat beban tinggi. Swap tidak mempercepat performa, tetapi mencegah crash mendadak.

5.4 Firewall UFW

```
sudo ufw default deny incoming
sudo ufw default allow outgoing

sudo ufw allow 22/tcp # SSH — WAJIB sebelum enable
sudo ufw allow 80/tcp # HTTP (redirect ke HTTPS)
sudo ufw allow 443/tcp # HTTPS
sudo ufw allow 81/tcp # NPM Admin — ditutup lagi setelah setup (Bab 7.4)

sudo ufw enable
sudo ufw status verbose
```

BASH

Peringatan Urutan Eksekusi

Jalankan `sudo ufw allow 22/tcp` sebelum `sudo ufw enable`. Jika terlewat, sesi SSH akan terputus dan VPS tidak dapat diakses kembali tanpa konsol out-of-band dari panel penyedia VPS.

5.5 Fail2Ban — Proteksi Brute-Force SSH

```
sudo systemctl enable fail2ban
sudo systemctl start fail2ban
sudo fail2ban-client status
sudo fail2ban-client status sshd
```

BASH

5.6 Autentikasi SSH Berbasis Key (Opsional, Direkomendasikan)

```
# Dari komputer lokal (bukan di VPS)
ssh-keygen -t ed25519 -f ~/.ssh/learnly_vps -C "learnly-admin"
ssh-copy-id -i ~/.ssh/learnly_vps.pub ubuntu@43.173.11.92

# Tes login tanpa password
ssh -i ~/.ssh/learnly_vps ubuntu@43.173.11.92

# Setelah berhasil, nonaktifkan password auth di VPS:
sudo nano /etc/ssh/sshd_config
# Ubah: PasswordAuthentication yes → PasswordAuthentication no
sudo systemctl reload sshd
```

BASH

5.7 Install Docker Engine & Compose Plugin (Resmi)

```
# Hapus versi lama bila ada
sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null

# Tambahkan repository resmi Docker
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Izinkan user ubuntu menjalankan docker tanpa sudo
sudo usermod -aG docker ubuntu
newgrp docker

docker --version
docker compose version
sudo systemctl enable docker
```

BASH

Checkpoint Bab 5

Pada titik ini: sistem ter-update, timezone WIB, swap 2 GB aktif, UFW mengizinkan hanya port 22/80/443/81, Fail2Ban melindungi SSH, dan Docker Engine + Compose plugin siap digunakan oleh user ubuntu tanpa sudo.

Struktur Direktori Server

6.1 Pembuatan Direktori

```
sudo mkdir -p /apps/nginx-proxy-manager
sudo mkdir -p /apps/learnly/scripts
sudo mkdir -p /backups/mysql
sudo mkdir -p /logs/learnly
sudo chown -R ubuntu:ubuntu /apps /backups /logs
```

BASH

6.2 Struktur Direktori Akhir

```
/
├─ apps/
│   ├─ nginx-proxy-manager/
│   │   ├─ docker-compose.yml ← konfigurasi NPM
│   │   └─ data/ ← data NPM (auto-generate)
│   └─ learnly/
│       ├─ letsencrypt/ ← sertifikat SSL (auto-generate)
│       └─ learnly/
│           ├─ docker-compose.yml ← konfigurasi utama Learnly
│           ├─ .env.production ← secrets & environment vars (jangan commit)
│           ├─ api/ ← source code backend (clone dari GitHub)
│           ├─ web/ ← source code frontend (clone dari GitHub)
│           └─ scripts/
│               └─ backup-db.sh ← skrip backup otomatis
├─ backups/
│   └─ mysql/ ← file .sql.gz hasil backup harian
└─ logs/
    └─ learnly/ ← log backup & deploy
```

6.3 Clone Repository

```
cd /apps/learnly
git clone https://github.com/USERNAME/learnly.git .

# Pastikan kedua folder terisi:
ls -la api/ web/
```

BASH

Verifikasi Setelah Clone

Struktur repositori nyata memiliki `api/` dan `web/` sebagai dua folder sejajar tanpa tooling monorepo (sesuai docs/03-tech-stack.md §2.5). Pastikan juga folder `api/prisma/` dan `api/seed-assets/` ikut ter-clone — keduanya dibutuhkan saat build (migration) dan seed demo.

Nginx Proxy Manager (Reverse Proxy & SSL)

Nginx Proxy Manager (NPM) berfungsi sebagai reverse proxy tunggal yang menangani terminasi TLS/SSL dan routing domain ke kontainer yang tepat, tanpa perlu mengedit konfigurasi Nginx manual.

7.1 Buat Docker Network External

```
docker network create proxy
```

BASH

Network `proxy` dipakai bersama oleh NPM dan kontainer `learnly_web` / `learnly_api` (lihat Bab 3.2).

7.2 Docker Compose NPM

```
nano /apps/nginx-proxy-manager/docker-compose.yml
```

BASH

```
services:
  app:
    image: 'jc21/nginx-proxy-manager:latest'
    container_name: nginx_proxy_manager
    restart: unless-stopped
    ports:
      - '80:80'
      - '81:81'
      - '443:443'
    volumes:
      - './data:/data'
      - './letsencrypt:/etc/letsencrypt'
    networks:
      - proxy

networks:
  proxy:
    external: true
```

YAML

7.3 Jalankan NPM & Login Pertama

```
cd /apps/nginx-proxy-manager
docker compose up -d
docker ps
```

BASH

- 1 Buka `http://43.173.11.92:81` di browser.
- 2 Login default: email `admin@example.com`, password `changeme`.
- 3 Sistem memaksa penggantian email & password saat login pertama — gunakan kredensial kuat yang unik.

7.4 Tutup Port 81 Setelah Konfigurasi Selesai

```
sudo ufw delete allow 81/tcp
sudo ufw status verbose
```

BASH

Akses Darurat ke Admin Panel Setelah Ditutup

Gunakan SSH tunnel bila sesekali perlu masuk kembali ke panel admin: `ssh -L 8081:localhost:81 ubuntu@43.173.11.92`, lalu akses `http://localhost:8081` dari komputer lokal.

Kontainerisasi Docker — Orkestrasi Learnly

File `docker-compose.yml` berikut mendefinisikan tiga layanan inti Learnly (database, backend, frontend). Berbeda dari `docker-compose.yml` pengembangan lokal yang hanya memuat MySQL (lihat komentar baris 1–2 pada file tersebut: *"Dev lokal saja: hanya MySQL. API & web dijalankan langsung dengan `npm run dev`"*), file ini adalah orkestrasi produksi lengkap.

8.1 File: `/apps/learnly/docker-compose.yml`

```

services:
# — DATABASE —————
mysql:
  image: mysql:8.4
  container_name: learnly_db
  restart: unless-stopped
  environment:
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    MYSQL_DATABASE: ${MYSQL_DATABASE}
    MYSQL_USER: ${MYSQL_USER}
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    TZ: Asia/Jakarta
  volumes:
    - mysql_data:/var/lib/mysql
  networks:
    - learnly-network
  mem_limit: 800m
  healthcheck:
    # F-06: kredensial WAJIB disertakan agar healthcheck tidak false-positive
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p${MYSQL_ROOT_PASSWORD}"]
    interval: 10s
    timeout: 5s
    retries: 10

# — BACKEND API —————
api:
  build:
    context: ./api
    dockerfile: Dockerfile
  container_name: learnly_api
  restart: unless-stopped
  env_file: .env.production
  environment:
    NODE_ENV: production
    PORT: 5000
    DATABASE_URL: >-
      mysql://${MYSQL_USER}:${MYSQL_PASSWORD}@learnly_db:3306/${MYSQL_DATABASE}
  volumes:
    # F-05: volume persisten WAJIB bila STORAGE_DRIVER=local, agar upload tidak hilang saat rebuild
    - learnly_uploads:/app/uploads
  depends_on:
    mysql:
      condition: service_healthy
  networks:
    - learnly-network
    - proxy
  mem_limit: 350m

# — FRONTEND WEB —————
web:
  build:
    context: ./web
    dockerfile: Dockerfile
  args:
    API_PROXY_TARGET: http://learnly_api:5000
  container_name: learnly_web
  restart: unless-stopped
  environment:
    NODE_ENV: production
    PORT: 3000
    NEXT_PUBLIC_API_BASE_URL: /api/v1
    API_PROXY_TARGET: http://learnly_api:5000
    NEXT_PUBLIC_MAP_TILE_URL: https://tile.openstreetmap.org/{z}/{x}/{y}.png
  depends_on:
    - api
  networks:
    - learnly-network
    - proxy
  mem_limit: 300m

volumes:
  mysql_data:
    name: learnly_mysql_data
  learnly_uploads:
    name: learnly_uploads_data

networks:
  learnly-network:
    driver: bridge

```

```
proxy:
  external: true
```

Perubahan Terhadap Draf Sebelumnya

Tiga perbaikan utama dibanding handbook lama: (1) healthcheck MySQL kini menyertakan kredensial root; (2) volume `learnly_uploads` ditambahkan dan dipasang ke `/app/uploads` sesuai `UPLOAD_DIR` pada `api/src/lib/storage.ts`; (3) `mem_limit` ditambahkan pada setiap service sesuai anggaran memori Bab 4.1 agar satu kontainer tidak dapat menghabiskan seluruh RAM VPS secara tidak terkendali.

8.2 Mitigasi Risiko Upload Lokal (F-05)

Volume `learnly_uploads` adalah mitigasi *minimum*, bukan solusi ideal. File yang tersimpan di volume Docker lokal tetap berada pada satu VPS tunggal — jika disk korup atau VPS hilang tanpa backup terpisah, file tetap berisiko hilang. Rekomendasi kuat tetap mengaktifkan:

```
STORAGE_DRIVER=cloudinary
CLOUDINARY_CLOUD_NAME=xxxxx
CLOUDINARY_API_KEY=xxxxxx
CLOUDINARY_API_SECRET=xxxxxx
```

ENV

Cloudinary free tier (25 GB bandwidth/bulan) sudah cukup untuk skala awal dan menghilangkan dependensi penyimpanan file pada VPS sepenuhnya — sesuai desain awal proyek ([docs/03-tech-stack.md](#) §2.4).

Frontend — Next.js 16 (Dockerfile & Konfigurasi)

9.1 Aktifkan Output Standalone

File asli `web/next.config.ts` belum memiliki `output: 'standalone'`. Konfigurasi ini wajib untuk image Docker ringan. Berikut versi final yang mempertahankan logika proxy same-origin yang sudah ada (lihat Bab 3.3) dan menambahkan baris `output`:

```
import type { NextConfig } from 'next';TYPESCRIPT  
  
// Proxy same-origin ke Learnly API: browser memanggil /api/v1/* di domain web,  
// Next.js meneruskannya ke API. Cookie refresh token jadi first-party.  
const apiBase = process.env.NEXT_PUBLIC_API_BASE_URL ?? '/api/v1';  
const proxyTarget = (  
  process.env.API_PROXY_TARGET ?? (process.env.NODE_ENV === 'production' ? '' : 'http://localhost:4000')  
)  
  .replace(/\/+$/, '');  
  
if (apiBase.startsWith('/') && !proxyTarget) {  
  throw new Error(  
    'API_PROXY_TARGET wajib diisi (mis. http://learnly_api:5000) karena NEXT_PUBLIC_API_BASE_URL relatif.',  
  );  
}  
  
const nextConfig: NextConfig = {  
  output: 'standalone', // ← WAJIB ditambahkan untuk Docker  
  async rewrites() {  
    if (!proxyTarget) return [];  
    return [  
      { source: '/api/v1/:path*', destination: `${proxyTarget}/api/v1/:path*` },  
      { source: '/uploads/:path*', destination: `${proxyTarget}/uploads/:path*` },  
    ];  
  },  
};  
  
export default nextConfig;
```

Wajib untuk Build Docker

Tanpa `output: 'standalone'`, direktori `.next/standalone` yang di-COPY pada Dockerfile tidak akan pernah dihasilkan, dan build Docker akan gagal pada tahap akhir dengan error "folder tidak ditemukan".

9.2 Dockerfile web/Dockerfile

```

# Stage 1: Install dependencies
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

# Stage 2: Build Next.js
FROM node:20-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .

ARG API_PROXY_TARGET
ENV API_PROXY_TARGET=${API_PROXY_TARGET}
ENV NEXT_PUBLIC_API_BASE_URL=/api/v1
ENV NEXT_TELEMETRY_DISABLED=1
ENV NODE_ENV=production

RUN npm run build

# Stage 3: Production Runner
FROM node:20-alpine AS runner
WORKDIR /app

ENV NODE_ENV=production
ENV PORT=3000
ENV NEXT_TELEMETRY_DISABLED=1

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static

USER nextjs
EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=10s --start-period=20s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000 || exit 1

CMD ["node", "server.js"]

```

DOCKERFILE

9.3 web/.dockerignore

```

node_modules
.next
.env
.env.*
*.log
.git
.gitignore
coverage

```

TEXT

Catatan: Folder public/ Tidak Ditemukan

Audit repositori menunjukkan web/ saat ini belum memiliki folder public/. Instruksi `COPY --from=builder /app/public ./public` pada Dockerfile tetap aman dijalankan meski folder kosong/tidak ada — Next.js akan tetap membuat direktori public kosong pada proses build jika diperlukan. Bila tim menambahkan aset statis (favicon, gambar) di kemudian hari, letakkan di web/public/ tanpa perlu mengubah Dockerfile.

9.4 Mengapa Multi-Stage Build

STAGE	FUNGSI	ESTIMASI UKURAN
deps	Instalasi <code>node_modules</code> saja (di-cache sebagai layer)	≈ 450–550 MB
builder	Build Next.js (<code>npm run build</code> , Turbopack)	≈ 750–900 MB
runner	Hanya file production yang dibutuhkan	≈ 110–140 MB ✓

Backend — Express.js 5 & Prisma (Dockerfile & Konfigurasi)

10.1 Skrip package.json yang Relevan

Dockerfile pada bab ini disusun agar persis mengikuti skrip yang sudah terdefinisi di `api/package.json`, tanpa menciptakan perintah baru:

SKRIP	ISI PERINTAH	DIPAKAI PADA STAGE
build	<code>prisma generate && tsc -p tsconfig.build.json</code>	builder
start	<code>prisma migrate deploy && node dist/scripts/seed.js && node dist/server.js</code>	runner (CMD)

Koreksi F-01 (Bab 2)

Perintah CMD Dockerfile di bawah ini **menjalankan skrip start apa adanya** melalui `npm run start`, bukan menulis ulang perintah migrate+server secara manual seperti handbook lama. Ini memastikan seed admin pertama (Bab 2, F-01 & F-02) selalu dijalankan otomatis pada setiap startup container — aman karena `seedBase()` bersifat idempotent (hanya membuat data yang belum ada).

10.2 Dockerfile api/Dockerfile

```
# Stage 1: Build TypeScript & Generate Prisma Client
FROM node:20-alpine AS builder
WORKDIR /app

COPY package.json package-lock.json ./
COPY prisma ./prisma/
RUN npm ci

COPY . .
RUN npm run build
# ^ menjalankan: prisma generate && tsc -p tsconfig.build.json (sesuai package.json)

# Stage 2: Production Runner
FROM node:20-alpine AS runner
WORKDIR /app

ENV NODE_ENV=production

COPY package.json package-lock.json ./
COPY prisma ./prisma/
RUN npm ci --omit=dev

COPY --from=builder /app/dist ./dist
COPY seed-assets ./seed-assets/

# F-09: prisma generate dijalankan DI DALAM image alpine ini,
# bukan di-copy mentah dari stage lain — binary engine Prisma harus cocok platform runtime.
RUN npx prisma generate

EXPOSE 5000

# F-04: healthcheck ke /health (root, lebih ringan — sesuai api/railway.json)
HEALTHCHECK --interval=30s --timeout=10s --start-period=30s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:5000/health || exit 1

# F-01: menjalankan skrip "start" asli dari package.json —
# migrate deploy → seed admin/master-data → start server
CMD ["npm", "run", "start"]
```

Mengapa Tidak Menyalin uploads/ ke Image

Direktori `api/uploads` sengaja tidak di-COPY ke image — direktori ini dibuat otomatis oleh `fs.mkdir(dir, { recursive: true })` pada `api/src/lib/storage.ts` saat file pertama diunggah, dan datanya dipersiskan melalui volume `learnly_uploads` (Bab 8.1), bukan dari dalam image.

10.3 api/.dockerignore

```
node_modules
dist
.env
.env.*
*.log
.git
.gitignore
coverage
uploads/
```

TEXT

10.4 Daftar Environment Variable Backend (Sesuai `api/src/config/env.ts`)

VARIABEL	WAJIB?	CONTOH NILAI PRODUKSI	KETERANGAN
<code>NODE_ENV</code>	Ya	<code>production</code>	Mengaktifkan validasi <code>SEED_ADMIN_PASSWORD</code> ≥ 12 karakter
<code>PORT</code>	Tidak (default 4000)	<code>5000</code>	Diselaraskan dengan port container
<code>DATABASE_URL</code>	Ya	<code>mysql://user:pass@learnly_db:3306/learnly_prod</code>	Harus diawali <code>mysql://</code> (divalidasi regex)
<code>JWT_ACCESS_SECRET</code>	Ya	string acak ≥ 32 karakter	Ditolak start jika < 32 karakter
<code>JWT_REFRESH_SECRET</code>	Ya	string acak ≥ 32 karakter, berbeda dari access	Idem
<code>CORS_ORIGIN</code>	Tidak (default <code>localhost:3000</code>)	<code>https://learnly.web.id</code>	Dipisah koma bila lebih dari satu origin (lihat F-08)
<code>COOKIE_SAME_SITE</code>	Tidak (default <code>lax</code>)	<code>lax</code>	Tetap <code>lax</code> karena proxy same-origin dipertahankan (Bab 3.3)
<code>TRUST_PROXY_HOPS</code>	Tidak (default 1)	<code>1</code>	Koreksi F-03 — hanya 1 proxy (NPM) di depan API pada topologi VPS
<code>API_PUBLIC_URL</code>	Tidak	<code>https://learnly.web.id</code>	Dipakai membentuk URL file bila <code>STORAGE_DRIVER=local</code>
<code>WEB_APP_URL</code>	Tidak (default <code>localhost:3000</code>)	<code>https://learnly.web.id</code>	Untuk link reset password di email
<code>STORAGE_DRIVER</code>	Tidak (default <code>local</code>)	<code>cloudinary</code> (rekendasi)	Lihat mitigasi F-05, Bab 8.2
<code>CLOUDINARY_CLOUD_NAME/API_KEY/API_SECRET</code>	Wajib jika <code>STORAGE_DRIVER=cloudinary</code>	sesuai akun Cloudinary	Divalidasi <code>superRefine</code> — start gagal jika kosong
<code>MAIL_DRIVER</code>	Tidak (default <code>log</code>)	<code>smtp</code> (opsional)	<code>log</code> = email dicetak ke console saja
<code>EMAIL_USER/EMAIL_APP_PASSWORD</code>	Wajib jika <code>MAIL_DRIVER=smtp</code>	App Password Gmail 16 karakter	Divalidasi format email
<code>SEED_ADMIN_EMAIL</code>	Tidak (default <code>admin@learnly.id</code>)	<code>admin@learnly.web.id</code>	Dibaca oleh <code>seed-base.ts</code> , bukan oleh <code>env.ts</code>
<code>SEED_ADMIN_PASSWORD</code>	Wajib di production	≥ 12 karakter, acak	Koreksi F-02 — startup gagal total bila kosong & <code>NODE_ENV=production</code>
<code>ALLOW_DEMO_SEED</code>	Tidak	<i>jangan diisi</i> / <code>false</code>	Hanya diisi <code>true</code> sementara untuk keperluan demo presentasi

Basis Data MySQL 8.4 & Strategi Backup

11.1 File .env.production — Versi Final Terkoreksi

```
nano /apps/learnly/.env.production
```

BASH

```
# ===== DATABASE =====
MYSQL_ROOT_PASSWORD=GantiPasswordRootYangKuat!2026#Unik
MYSQL_DATABASE=learnly_prod
MYSQL_USER=learnly_user
MYSQL_PASSWORD=GantiPasswordUserYangKuat!2026#Unik

# ===== AUTH JWT (≥32 karakter, generate dengan perintah di bawah) =====
JWT_ACCESS_SECRET=GANTI_DENGAN_RANDOM_STRING_48_BYTE_HEX
JWT_REFRESH_SECRET=GANTI_DENGAN_RANDOM_STRING_48_BYTE_HEX_BERBEDA

# ===== SERVER =====
NODE_ENV=production
PORT=5000
CORS_ORIGIN=https://learnly.web.id,https://www.learnly.web.id
COOKIE_SAME_SITE=lax
# Koreksi F-03: HANYA 1 hop (NPM) pada topologi VPS — BUKAN 2
TRUST_PROXY_HOPS=1
API_PUBLIC_URL=https://learnly.web.id
WEB_APP_URL=https://learnly.web.id

# ===== FILE STORAGE (rekendasi: cloudinary — lihat Bab 8.2) =====
STORAGE_DRIVER=local
# STORAGE_DRIVER=cloudinary
# CLOUDINARY_CLOUD_NAME=your-cloud-name
# CLOUDINARY_API_KEY=your-api-key
# CLOUDINARY_API_SECRET=your-api-secret

# ===== EMAIL (opsional) =====
MAIL_DRIVER=log
# MAIL_DRIVER=smtp
# GMAIL_USER=your-account@gmail.com
# GMAIL_APP_PASSWORD=your-16-char-app-password

# ===== SEED ADMIN (KRITIS — koreksi F-02, WAJIB diisi) =====
# Tanpa dua baris ini, container API akan CRASH LOOP saat start pertama
# karena NODE_ENV=production mewajibkan password seed admin.
SEED_ADMIN_EMAIL=admin@learnly.web.id
SEED_ADMIN_PASSWORD=GantiPasswordAdminMin12Karakter!2026

# ===== DATA DEMO (biarkan kosong/false di production) =====
# ALLOW_DEMO_SEED=true
```

ENV

Jangan Commit File Ini

Pastikan .env.production tercantum pada .gitignore repositori (pola .env* dengan pengecualian !.env.example sudah ada di root .gitignore proyek). File ini hanya boleh berada di server, tidak pernah di Git.

11.2 Generate Secret JWT & Password

```
# Generate JWT_ACCESS_SECRET
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"

# Jalankan sekali lagi untuk JWT_REFRESH_SECRET (harus berbeda)
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

BASH

11.3 Skrip Backup Otomatis

```
nano /apps/learnly/scripts/backup-db.sh
```

BASH

```
#!/bin/bash
# =====
# Learnly – Backup Otomatis MySQL
# Backup harian dengan retensi 7 hari
# =====
set -e

BACKUP_DIR="/backups/mysql"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="$BACKUP_DIR/learnly_backup_${TIMESTAMP}.sql.gz"
RETENTION_DAYS=7
LOG_FILE="/logs/learnly/backup.log"

mkdir -p "$BACKUP_DIR" "$(dirname "$LOG_FILE")"

ENV_FILE="/apps/learnly/.env.production"
if [ ! -f "$ENV_FILE" ]; then
    echo "[$(date)] ERROR: $ENV_FILE tidak ditemukan!" | tee -a "$LOG_FILE"
    exit 1
fi
export $(grep -E '^(MYSQL_USER|MYSQL_PASSWORD|MYSQL_DATABASE)= ' "$ENV_FILE" | xargs)

echo "[$(date)] Memulai backup database..." >> "$LOG_FILE"

if ! docker exec learnly_db mysqldump \
    -u"$MYSQL_USER" -p"$MYSQL_PASSWORD" "$MYSQL_DATABASE" \
    --single-transaction --quick --lock-tables=false \
    | gzip > "$BACKUP_FILE"; then
    echo "[$(date)] ERROR: dump database gagal!" >> "$LOG_FILE"
    rm -f "$BACKUP_FILE"
    exit 1
fi

if [ -s "$BACKUP_FILE" ]; then
    SIZE=$(du -h "$BACKUP_FILE" | cut -f1)
    echo "[$(date)] Backup SUKSES: $BACKUP_FILE ($SIZE)" >> "$LOG_FILE"
else
    echo "[$(date)] ERROR: file backup kosong!" >> "$LOG_FILE"
    rm -f "$BACKUP_FILE"
    exit 1
fi

DELETED=$(find "$BACKUP_DIR" -type f -name "*.sql.gz" -mtime +$RETENTION_DAYS -delete -print | wc -l)
echo "[$(date)] Cleanup: $DELETED backup lama dihapus." >> "$LOG_FILE"
```

```
chmod +x /apps/learnly/scripts/backup-db.sh
/apps/learnly/scripts/backup-db.sh # tes manual
cat /logs/learnly/backup.log
```

11.4 Cron Job Harian (02:00 WIB)

```
crontab -e
```

```
0 2 * * * /apps/learnly/scripts/backup-db.sh
```

11.5 Prosedur Restore dari Backup

```
ls -lah /backups/mysql/

gunzip < /backups/mysql/learnly_backup_YYYYMMDD_HHMMSS.sql.gz \
| docker exec -i learnly_db mysql -u learnly_user -p'PASSWORD' learnly_prod
```

Rekomendasi Lanjutan: Backup Off-Site

Skip di atas menyimpan backup pada disk yang sama dengan VPS produksi — jika VPS hilang/korup total, backup lokal juga ikut hilang. Untuk mitigasi lanjutan (di luar scope wajib dokumen ini), pertimbangkan menyalin file .sql.gz terbaru ke storage eksternal (misalnya Cloudinary raw upload, atau object storage lain) setiap selesai backup harian.

Domain, DNS & Sertifikat SSL/TLS

12.1 Konfigurasi DNS `learnly.web.id`

Login ke panel DNS registrar domain `learnly.web.id`, lalu tambahkan record berikut yang mengarah ke IP publik VPS `43.173.11.92`:

TIPE	HOST	MENGARAH KE	TTL
A	@	<code>43.173.11.92</code>	3600
A	<code>api</code>	<code>43.173.11.92</code>	3600
A	<code>www</code>	<code>43.173.11.92</code>	3600

```
# Verifikasi propagasi DNS dari VPS
sudo apt install dnsutils -y
dig learnly.web.id +short
dig api.learnly.web.id +short
# Harus menampilkan 43.173.11.92
```

BASH

12.2 Proxy Host NPM — Frontend

FIELD (TAB DETAILS)	ISI
Domain Names	<code>learnly.web.id</code> , <code>www.learnly.web.id</code>
Scheme	<code>http</code>
Forward Hostname / IP	<code>learnly_web</code>
Forward Port	<code>3000</code>
Cache Assets / Block Common Exploits / Websockets	Semua dicentang

FIELD (TAB SSL)	ISI
SSL Certificate	Request a new SSL Certificate (Let's Encrypt)
Force SSL / HTTP/2 Support	Dicentang
Email	email operasional tim

12.3 Proxy Host NPM — Backend API

FIELD (TAB DETAILS)	ISI
Domain Names	<code>api.learnly.web.id</code>
Scheme	<code>http</code>
Forward Hostname / IP	<code>learnly_api</code>
Forward Port	<code>5000</code>

Fungsi Domain API Publik (Lihat Bab 3.3)

Browser web app Learnly **tidak** memanggil `api.learnly.web.id` secara langsung pada operasi normal — semua request melewati proxy same-origin `learnly.web.id/api/v1/*`. Domain API publik tetap dikonfigurasi untuk kebutuhan pengujian Postman dan observability independen (Bab 14).

12.4 Jalankan Seluruh Kontainer

```
cd /apps/learnly
docker compose --env-file .env.production up -d --build

docker ps
docker logs -f learnly_api
docker logs -f learnly_web
```

BASH

12.5 Verifikasi Live

- Frontend: `https://learnly.web.id`
- Health check backend (root): `https://api.learnly.web.id/health`
- Health check backend (v1): `https://api.learnly.web.id/api/v1/health`

Auto-Renew SSL

Sertifikat Let's Encrypt berlaku 90 hari dan diperpanjang otomatis oleh NPM — tidak dibutuhkan intervensi manual, selama port 80 tetap terbuka untuk proses validasi ACME HTTP-01.

CI/CD — GitHub Actions

Audit repositori tidak menemukan direktori `.github/workflows/` — pipeline CI/CD berikut dibuat baru, dirancang agar setiap `git push origin main` memicu deploy otomatis ke VPS melalui SSH.

13.1 Generate SSH Key Khusus CI/CD

```
# Di VPS (user ubuntu)
ssh-keygen -t ed25519 -C "github-actions-deploy" -f ~/.ssh/github_actions -N ""
cat ~/.ssh/github_actions.pub >> ~/.ssh/authorized_keys
cat ~/.ssh/github_actions    # copy private key ini ke GitHub Secrets
```

BASH

13.2 GitHub Repository Secrets

NAMA SECRET	NILAI
VPS_HOST	43.173.11.92
VPS_USERNAME	ubuntu
VPS_SSH_KEY	Isi lengkap <code>cat ~/.ssh/github_actions</code> (private key)
VPS_PORT	22

13.3 .github/workflows/deploy.yml

```
name: Deploy Learnly to Production VPS

on:
  push:
    branches:
      - main

jobs:
  deploy:
    name: Deploy to VPS
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Deploy via SSH
        uses: appleboy/ssh-action@v1.0.3
        with:
          host: ${ secrets.VPS_HOST }
          username: ${ secrets.VPS_USERNAME }
          key: ${ secrets.VPS_SSH_KEY }
          port: ${ secrets.VPS_PORT }
          script: |
            set -e
            echo "Deploy dimulai: $(date)"
            cd /apps/learnly

            git pull origin main

            docker compose --env-file .env.production \
              up -d --build --remove-orphans

            docker image prune -f

            echo "Deploy selesai: $(date)"
            docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

YAML

Developer
git push origin main

→

GitHub Actions
trigger workflow

→

SSH ke VPS
git pull + docker compose up

→

Live
learnly.web.id

Keterbatasan CI/CD yang Disengaja

Pipeline ini sengaja **tidak** menyertakan automated testing (`npm run test`, `npm run typecheck`) pada tahap awal untuk menjaga kesederhanaan. Rekomendasi: tambahkan job terpisah yang menjalankan `npm run lint && npm run typecheck && npm run test` sebelum job `deploy` dieksekusi, begitu kapasitas tim memungkinkan.

Observability, Troubleshooting & Kesiapan Produksi

14.1 Status & Resource Kontainer

```
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
docker stats --no-stream
```

BASH

14.2 Streaming Log

```
docker logs -f learnly_api
docker logs -f learnly_web
docker logs -f learnly_db
docker logs -f nginx_proxy_manager
docker logs --tail 100 learnly_api
```

BASH

14.3 Verifikasi TRUST_PROXY_HOPS (Koreksi F-03)

Setelah deploy, verifikasi bahwa Express membaca IP klien asli dengan benar, bukan IP internal Docker NPM:

```
# Masuk ke container API, cek header yang diterima dari NPM
docker exec -it learnly_api sh -c "echo \${TRUST_PROXY_HOPS}"
# Harus menampilkan: 1

# Uji dari luar – lakukan beberapa kali login salah untuk memicu rate-limit
# dan pastikan hanya IP Anda sendiri yang terkena limit, bukan semua pengguna.
```

BASH

14.4 Inspeksi Jaringan Docker

```
docker network ls
docker network inspect proxy
docker network inspect learnly-network
```

BASH

14.5 Log Rotation (Mencegah Disk Penuh — Bab 4.2)

Docker secara default tidak membatasi ukuran log kontainer. Tambahkan konfigurasi berikut di `/etc/docker/daemon.json` untuk mencegah disk penuh:

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

JSON

```
sudo systemctl restart docker
```

BASH

14.6 Debug & Masuk ke Dalam Kontainer

```
docker exec -it learnly_api sh
docker exec -it learnly_api npx prisma migrate deploy
docker exec -it learnly_api node dist/scripts/seed.js
docker exec -it learnly_db mysql -u learnly_user -p learnly_prod
```

BASH

14.7 Restart & Rebuild

```
docker compose --env-file .env.production restart api
docker compose --env-file .env.production up -d --build api
docker compose --env-file .env.production up -d --build
docker compose --env-file .env.production down
# PERINGATAN: menghapus volume = data database & upload HILANG
docker compose --env-file .env.production down -v
```

BASH

14.8 Restore Database

```
gunzip < /backups/mysql/learnly_backup_YYYYMMDD_HHMMSS.sql.gz \
| docker exec -i learnly_db mysql -u learnly_user -p'PASSWORD' learnly_prod
```

BASH

14.9 Tabel Troubleshooting Umum

SIMPTOM	KEMUNGKINAN SEBAB	TINDAKAN
Container <code>learnly_api</code> restart loop	F-02: <code>SEED_ADMIN_PASSWORD</code> kosong/<12 karakter di production	Isi <code>.env.production</code> sesuai Bab 11.1, lalu <code>docker compose up -d --build api</code>
Login admin gagal "user not found" setelah deploy pertama	F-01: start command belum menjalankan seed	Pastikan Dockerfile memakai <code>CMD ["npm", "run", "start"]</code> (Bab 10.2)
File upload hilang setelah deploy baru	F-05: volume uploads tidak dipasang	Tambahkan volume <code>learnly_uploads</code> (Bab 8.1) atau migrasi ke Cloudinary
Rate-limit terasa salah sasaran (satu user kena limit semua)	F-03: <code>TRUST_PROXY_HOPS</code> tidak sesuai jumlah proxy nyata	Set ke <code>1</code> untuk topologi VPS+NPM (Bab 11.1), verifikasi Bab 14.3
MySQL healthcheck selalu "healthy" walau password salah	F-06: healthcheck tanpa kredensial	Gunakan healthcheck final Bab 8.1 yang menyertakan <code>-u root -p</code>
Build Next.js gagal "standalone folder not found"	<code>output: 'standalone'</code> belum diset	Tambahkan ke <code>next.config.ts</code> sesuai Bab 9.1
VPS kehabisan disk	Log Docker tak terbatas / backup tak terretensi	Terapkan log rotation (14.5), cek retensi backup (Bab 11.3)
MySQL/API di-kill tiba-tiba oleh OS	OOM — swap tidak aktif	Verifikasi <code>free -h</code> , pastikan swap 2 GB aktif (Bab 5.3)

14.10 Checklist Kesiapan Produksi

Keamanan & Sistem Operasi

- ☐ OS Ubuntu ter-update (`apt update && apt upgrade`)
- ☐ Timezone diset ke `Asia/Jakarta`
- ☐ Swap 2 GB aktif dan permanen (`free -h` , `/etc/fstab`)
- ☐ UFW aktif; hanya port 22, 80, 443 terbuka ke publik
- ☐ Port 81 (NPM admin) sudah ditutup setelah setup
- ☐ Fail2Ban aktif melindungi SSH
- ☐ Log rotation Docker terkonfigurasi (`daemon.json`)

Docker & Kontainer

- ☐ Docker Engine & Compose plugin resmi (bukan `docker.io`)
- ☐ Network `proxy` (external) dibuat
- ☐ Semua kontainer *running*: NPM, `learnly_web` , `learnly_api` , `learnly_db`
- ☐ Volume `learnly_mysql_data` dan `learnly_uploads` terpasang persisten
- ☐ `mem_limit` diterapkan pada setiap service (Bab 8.1)

Environment & Koreksi Audit (Bab 2)

- ☐ `TRUST_PROXY_HOPS=1` (bukan 2) — koreksi F-03
- ☐ `SEED_ADMIN_EMAIL` & `SEED_ADMIN_PASSWORD` (≥ 12 karakter) terisi — koreksi F-02
- ☐ Dockerfile API memakai `CMD ["npm", "run", "start"]` — koreksi F-01
- ☐ Healthcheck MySQL menyertakan kredensial root — koreksi F-06
- ☐ `JWT_ACCESS_SECRET` & `JWT_REFRESH_SECRET` masing-masing ≥ 32 karakter dan berbeda

Domain & SSL

- ☐ DNS A record `learnly.web.id` & `api.learnly.web.id` mengarah ke `43.173.11.92`
- ☐ SSL Let's Encrypt aktif pada kedua domain, force HTTPS redirect
- ☐ `https://learnly.web.id` menampilkan halaman web
- ☐ `https://api.learnly.web.id/health` merespons `200 OK`

Basis Data & Backup

- ☐ Prisma migration (`migrate deploy`) berjalan sukses saat startup
- ☐ Admin pertama berhasil login dengan kredensial `SEED_ADMIN_*`
- ☐ `backup-db.sh` teruji manual dan menghasilkan file `.sql.gz` valid
- ☐ Cron job backup harian terpasang (`crontab -l`)
- ☐ Prosedur restore sudah pernah diuji minimal satu kali di lingkungan staging

CI/CD

- ☐ SSH key khusus CI/CD ter-generate & terpasang di `authorized_keys`
- ☐ GitHub Secrets (`VPS_HOST` , `VPS_USERNAME=ubuntu` , `VPS_SSH_KEY` , `VPS_PORT`) terkonfigurasi
- ☐ Push percobaan ke `main` berhasil memicu deploy otomatis

Skalabilitas & Kapasitas (Bab 4)

- ☐ Monitoring `docker stats` menunjukkan penggunaan RAM $< 80\%$ pada beban normal
- ☐ Rencana trigger upgrade RAM/VPS sudah dipahami tim (Bab 4.4)
- ☐ Keputusan `STORAGE_DRIVER` (local+volume vs Cloudinary) sudah final dan terdokumentasi

Fungsional Akhir

- ☐ Login, register, dan fitur pencarian tutor berbasis lokasi berfungsi normal
- ☐ Upload bukti transfer & verifikasi admin berjalan end-to-end
- ☐ Polling status booking (refresh 5 detik) berjalan tanpa error di console browser

Status Akhir

Setelah seluruh item checklist di atas terverifikasi, **learnly.web.id** dinyatakan siap menerima trafik publik sesuai batasan skala yang dijelaskan pada Bab 4. Lakukan pemantauan intensif pada 48 jam pertama pasca go-live.

14.11 Lampiran A — Peta Modul API (Referensi Cepat)

Berdasarkan `api/src/app.ts`, seluruh endpoint dipasang di bawah prefix `/api/v1` (serta `/health` di root):

PREFIX	MODUL	FUNGSI RINGKAS
<code>/auth</code>	auth	Register, login, refresh, logout, forgot/reset password
<code>/users</code> , <code>/learners</code> , <code>/addresses</code>	users	Profil akun, data anak/pelajar, alamat pembelajaran
<code>/tutors</code>	tutors	Profil tutor, sertifikasi, wilayah layanan, jadwal, earnings
<code>/search</code>	search	Pencarian tutor (geospasial) & kursus
<code>/bookings</code>	bookings	Pemesanan sesi, status tracking, check-in/out
<code>/courses</code> , <code>/enrollments</code> , <code>/lessons</code> , <code>/assignments</code>	courses	Kursus daring, modul, materi, tugas
<code>/payments</code>	payments	Upload bukti transfer, verifikasi admin
<code>/reviews</code>	reviews	Rating & ulasan tutor/kursus
<code>/notifications</code>	notifications	Notifikasi in-app (dipoll bersamaan status)
<code>/settings</code>	settings	Pengaturan platform (biaya layanan, QRIS)
<code>/admin</code>	admin	Verifikasi tutor/pembayaran, dispute, master data, dashboard

14.12 Lampiran B — Rencana Mitigasi Risiko (Roadmap)

RISIKO	PROBABILITAS	DAMPAK	MITIGASI
OOM pada beban puncak (Bab 4.1)	Sedang-Tinggi	Tinggi	Swap aktif, <code>mem_limit</code> per service, monitor <code>docker stats</code> , upgrade RAM bila sustained >80%
Kehilangan file upload lokal	Sedang	Tinggi	Volume <code>learnly_uploads</code> (mitigasi minimum) + migrasi Cloudinary (mitigasi penuh, Bab 8.2)
Kegagalan backup tidak terdeteksi	Rendah-Sedang	Kritis	Monitoring <code>/logs/learnly/backup.log</code> berkala, uji restore periodik
VPS tunggal sebagai SPOF	Rendah	Kritis	Backup off-site (Bab 11.5), dokumentasi disaster recovery, pertimbangkan snapshot VPS berkala dari panel penyedia
Sertifikat SSL gagal renew	Rendah	Sedang	Auto-renew NPM aktif; pastikan port 80 tidak pernah ditutup permanen
Rate-limit tidak efektif pada scale-out	Rendah (saat ini)	Sedang	Dicatat sebagai batasan desain (F-07); migrasi ke Redis store hanya saat API benar-benar di-scale > 1 instance

Learnly

Production Deployment Handbook — Edisi Audit & Revisi 3.0

Dokumen No. LRN-OPS-HB-02 · Diterbitkan 8 Oktober 2026

Disusun berdasarkan audit langsung terhadap kode sumber repositori `learnly`

Target Infrastruktur · VPS 2 vCPU / 2 GB RAM / 40 GB SSD · `learnly.web.id`

Klasifikasi: Internal — Tim Teknik · Dokumen Terkendali